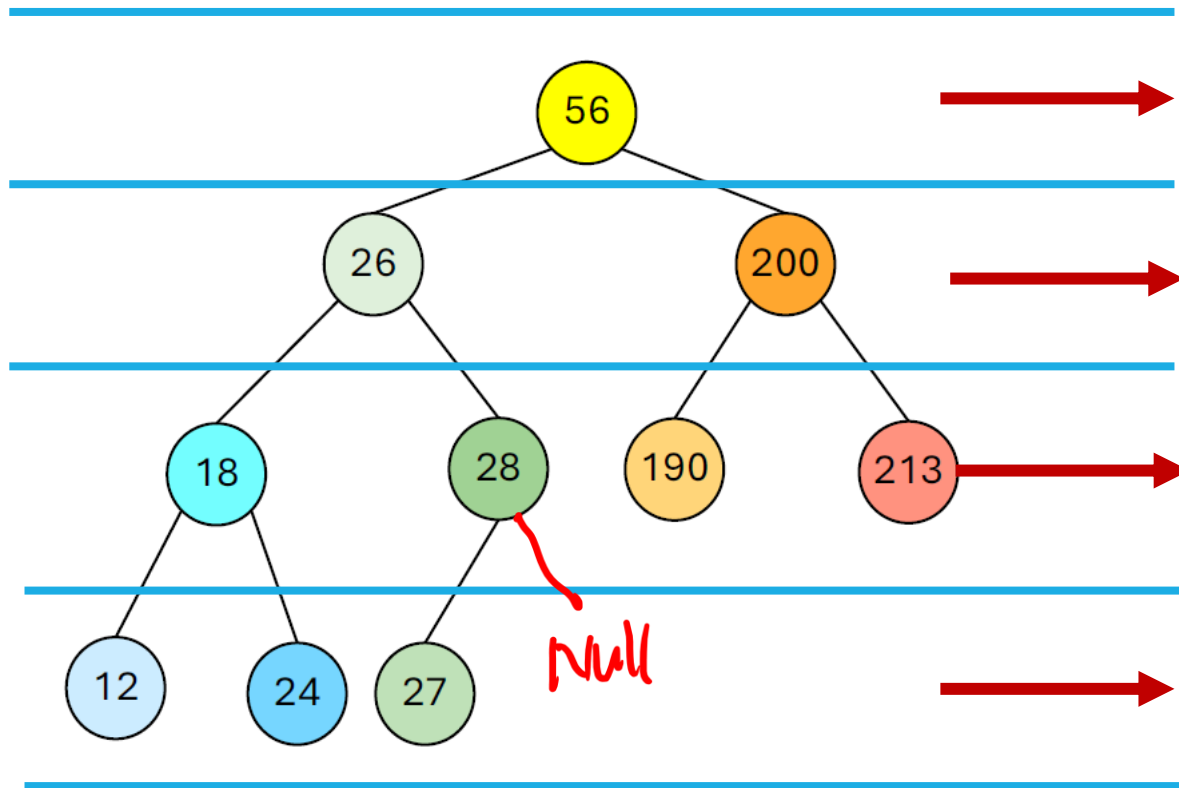


**BFS, DFS**



# BFS (Breadth-First Search)- Tree



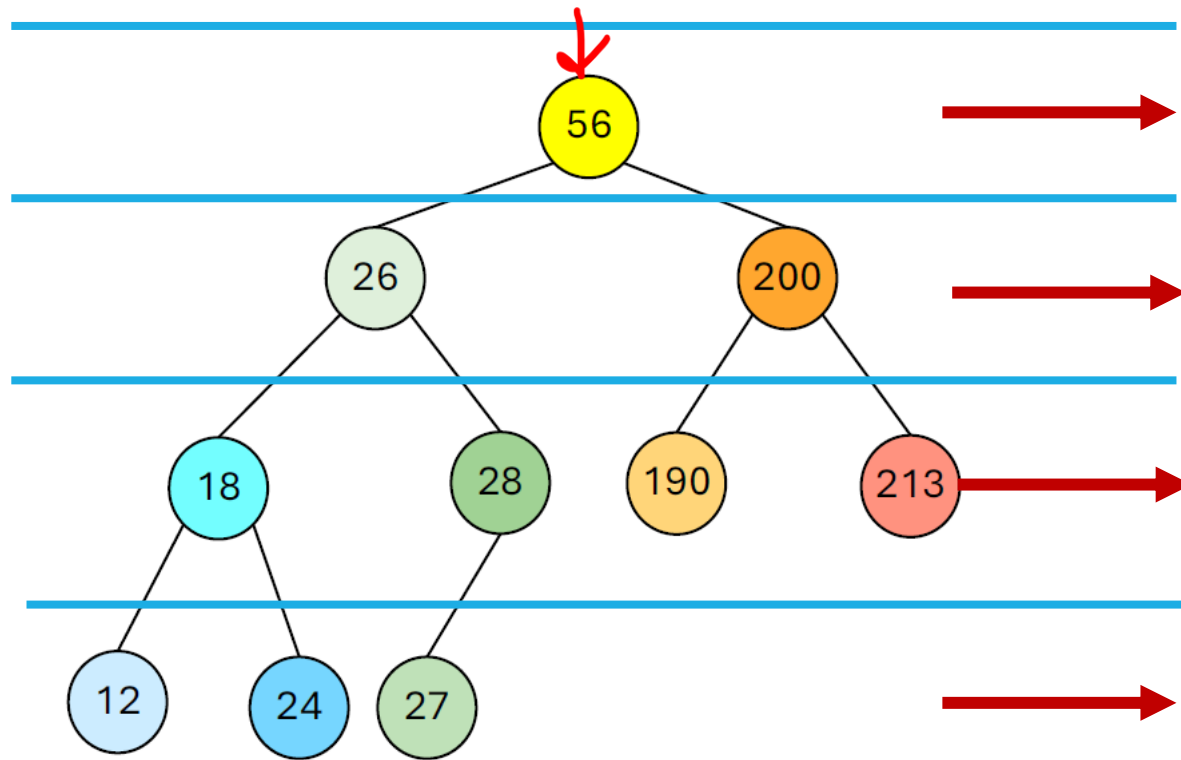
**BFS:** Visit nodes **level by level**—  
process all nodes at distance  $d$  from  
the start before any at distance  $d+1$ .

$56 \rightarrow 200(56), 26(56)$   
level 0                      level 1

$\rightarrow 213(200), 190(200), 28(26),$

$18(26) \rightarrow 27(28), 24(18), 12(18)$   
level 2                      level 3

# BFS (Breadth-First Search)- Tree



Q 56  
0

Q 26 200

dequeue 56 enqueue 26, 200

Q 200 18 28

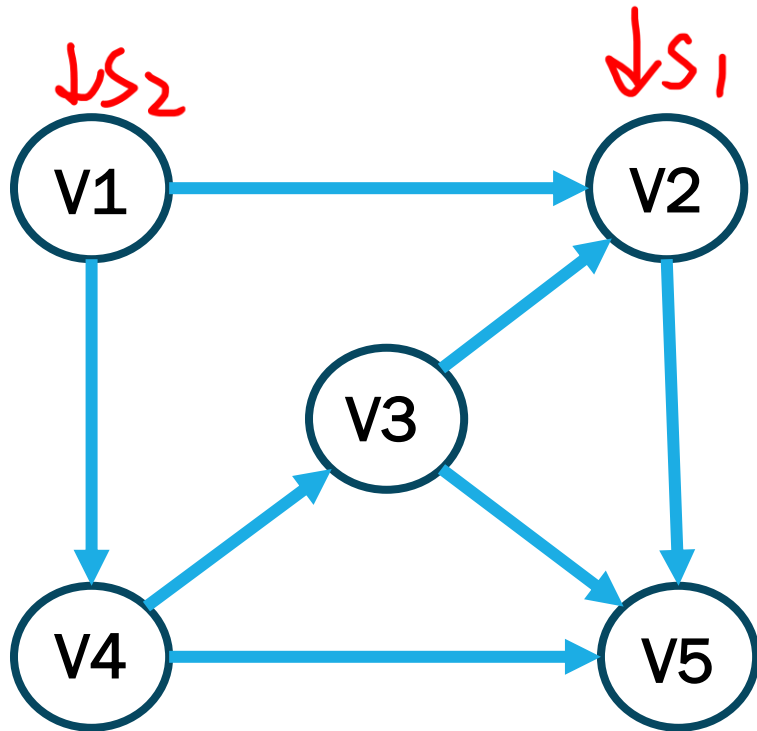
dequeue 26 enqueue 18, 28

Q 18 28 190 213  
2 2 2 2  
dequeue 200 enqueue 190, 213

Q 28 190 213 12 24  
2 2 2 3 3  
dequeue 18 enqueue 12, 24

Q 190 213 12 24 27  
2 2 3 3 3  
dequeue 28 enqueue 27

# BFS Graph-directed

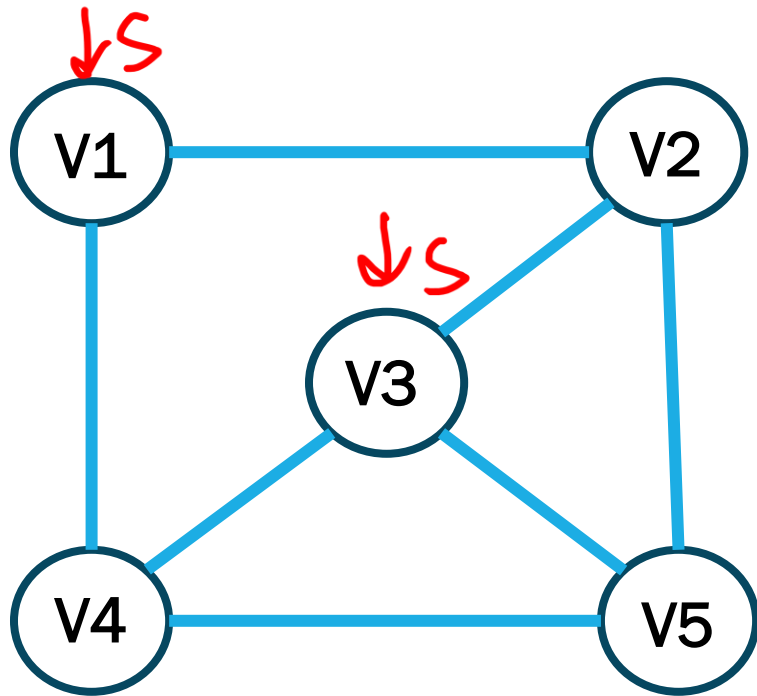


$V_1 \rightarrow V_2(V_1), V_4(V_1) \rightarrow V_5(V_2), V_3(V_4)$   
 0                      1                      1                      2                      2

$V_1 \rightarrow V_4(V_1), V_2(V_1) \rightarrow V_3(V_4), V_5(V_4)$   
 0                      1                      1                      2                      2

$V_2 \rightarrow V_5$                        $V_1 \rightarrow V_4 \rightarrow V_3$   
 0                      1

# BFS Graph-undirected



$V_1 \rightarrow V_2(V_1), V_4(V_1) \rightarrow V_3(V_2), V_5(V_2)$   
0                      1                      2                      2

$V_4 \rightarrow V_4(V_1), V_2(V_1) \rightarrow V_3(V_4), V_5(V_4)$   
0                      1                      2                      2

$V_3 \rightarrow V_2(V_3), V_4(V_3), V_5(V_3) \rightarrow V_1(V_2)$   
0                      1                      1                      2

# BFS implementation (Queue)

```
# Level-order (BFS)
def bfs_level_order(root):
    if not root:
        return []
    q = deque([root])
    order = []
    while q:
        u = q.popleft()
        order.append(u.key)
        if u.left:
            q.append(u.left)
        if u.right:
            q.append(u.right)
    return order
```

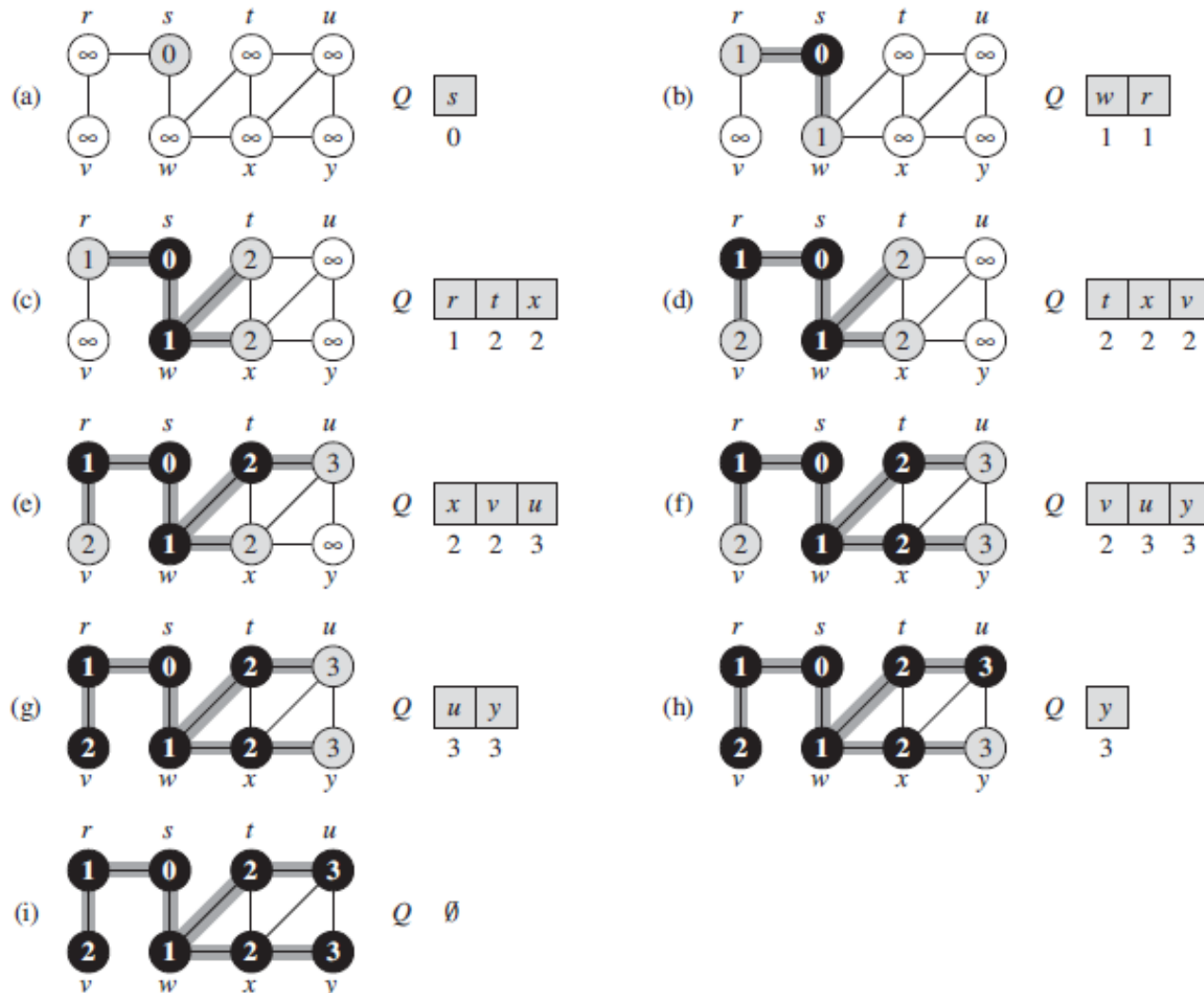
BFS( $G, s$ )

```
1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each vertex  $v$  in  $G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 
```

**white** = undiscovered;  
**gray** = discovered/enqueued;  
**black** = finished.  
**pi** to record the node before current node

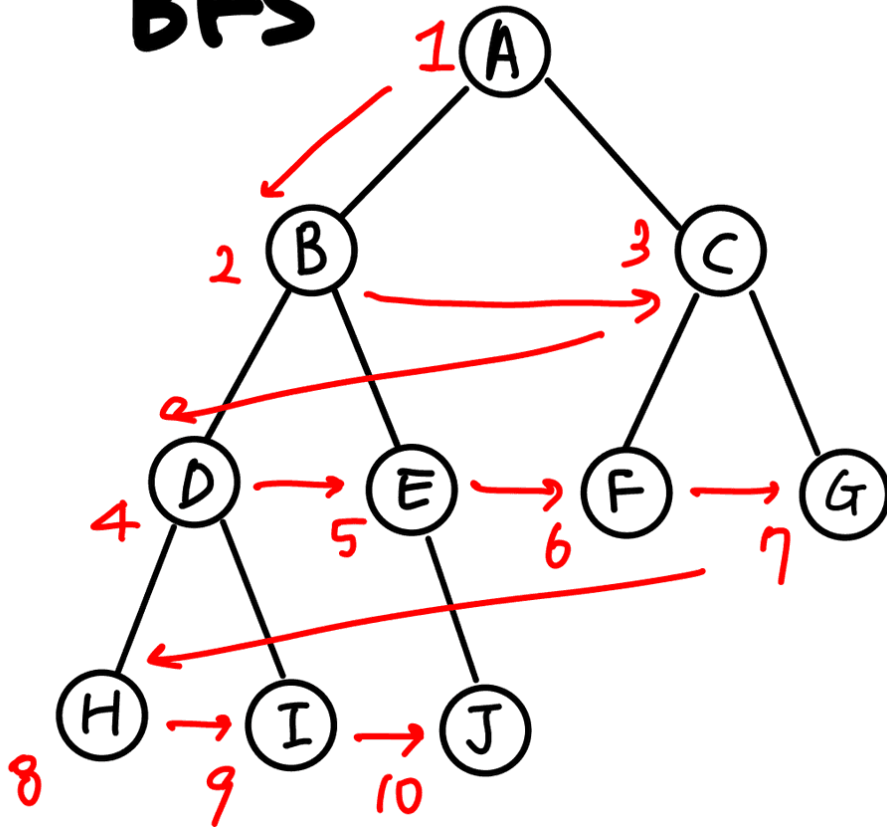
// search the neighbors of  $u$   
// is  $v$  being discovered now?  
//  $v$  is now on the frontier  
//  $u$  is now behind the frontier

# BFS implementation (Queue)

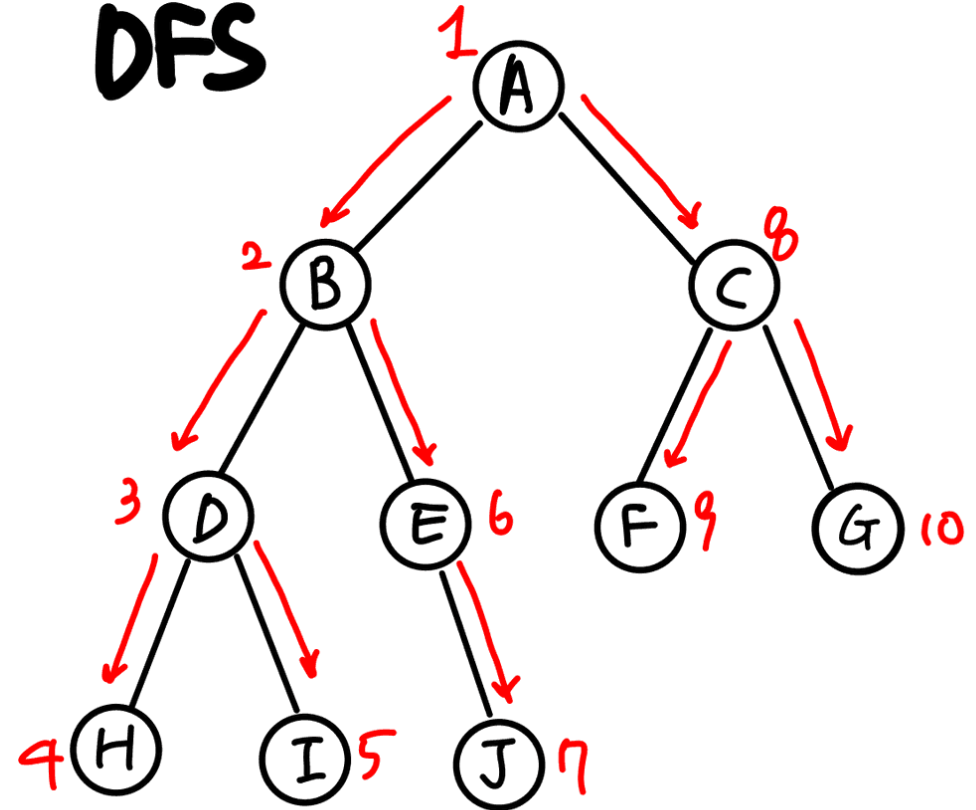


# BFS VS DFS

## BFS

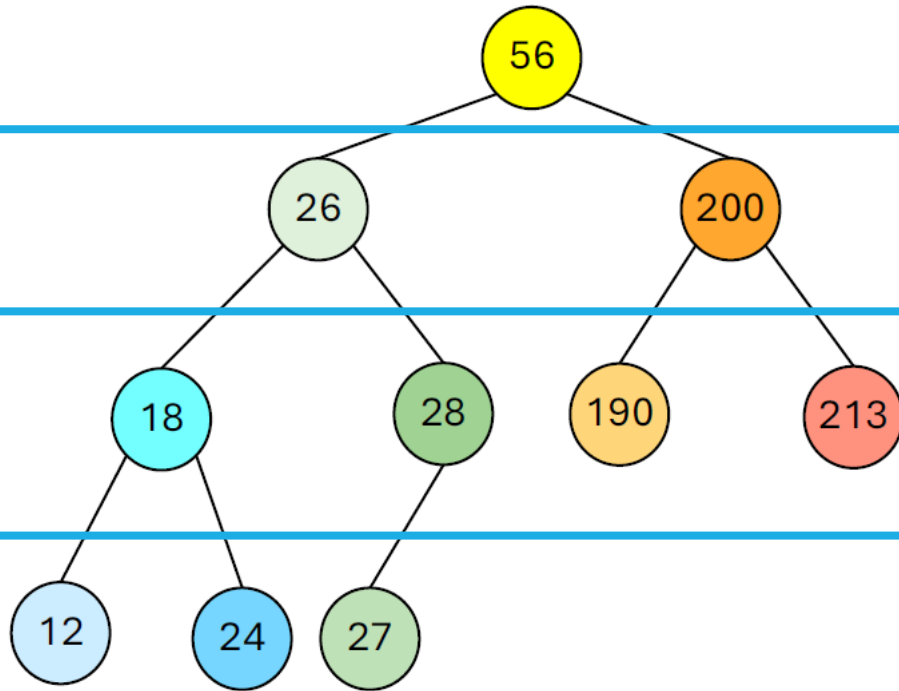


## DFS



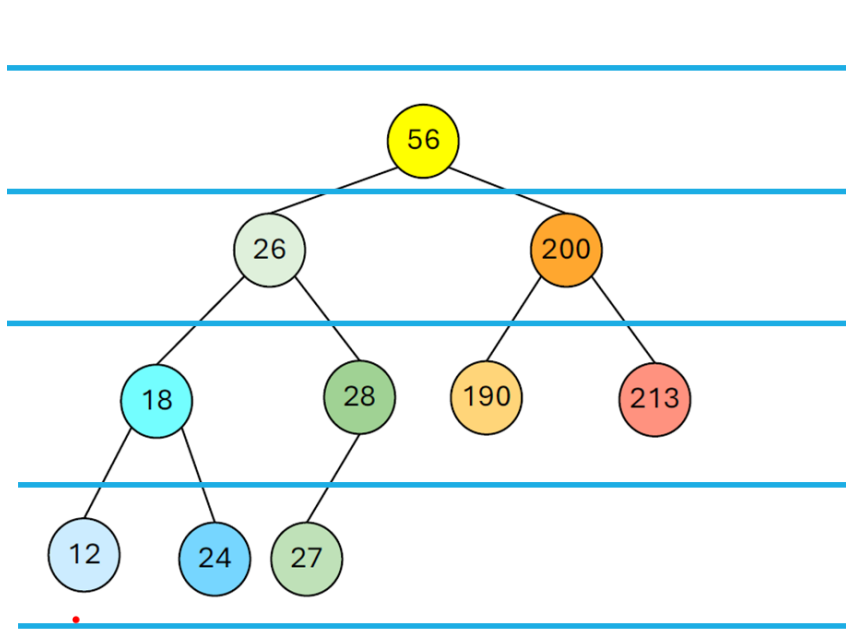


# DFS (Depth-First Search)- Tree



**DFS:** Go **as far as possible along one branch**; when you can't continue, **backtrack** and try the next branch.

# DFS (Depth-First Search)- Tree

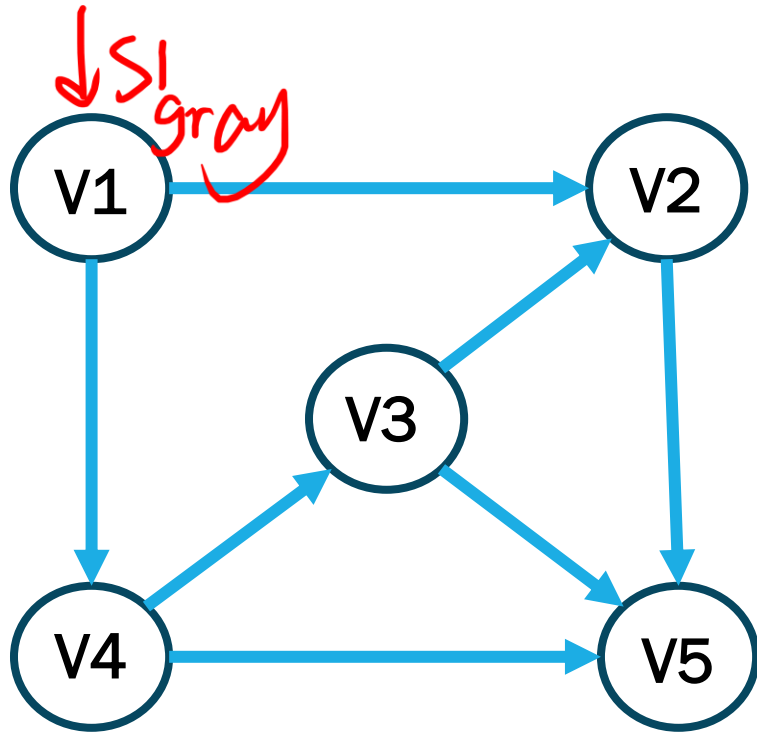


- Preorder, inorder, and postorder all use a depth-first (DFS) strategy.
- They differ only in when the current node is processed:
  - preorder (before children),
  - inorder (between left and right—binary trees only),
  - postorder (after both children).
- Here we consider “DFS order” as first-time discovery, that matches preorder.

middle-left-right

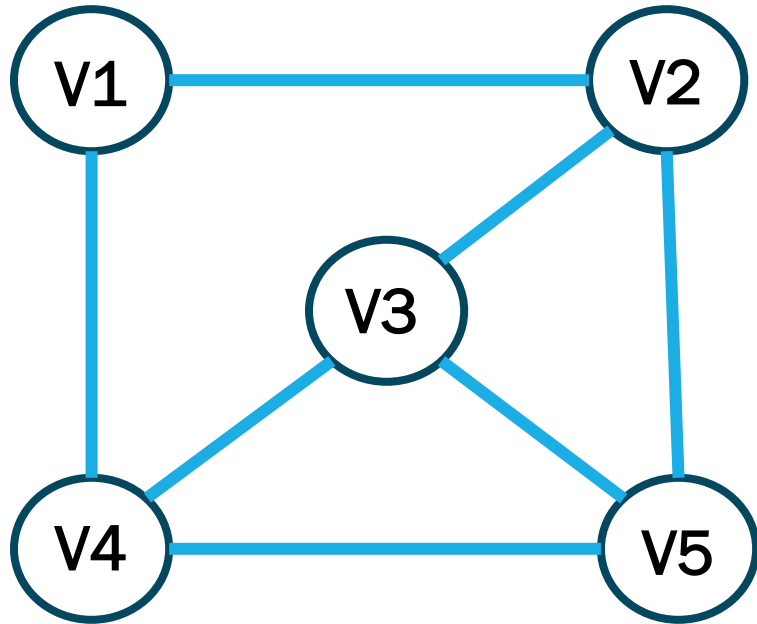
56 → 26 → 18 → 12 → 24 → 28 → 27 → 200 → 190 → 213  
[56] + [26] + [18] + [12]

## DFS Graph-directed



$V_1(\text{gray}) \rightarrow V_2(\text{gray}) \rightarrow V_5(\text{gray})$   
 $\rightarrow V_5(\text{black}) \rightarrow V_2(\text{black}) \rightarrow V_1$   
 $\rightarrow V_4(\text{gray}) \rightarrow V_3(\text{gray})$   
 $\rightarrow V_3(\text{black}) \rightarrow V_4(\text{black})$   
 $\rightarrow V_1(\text{black})$

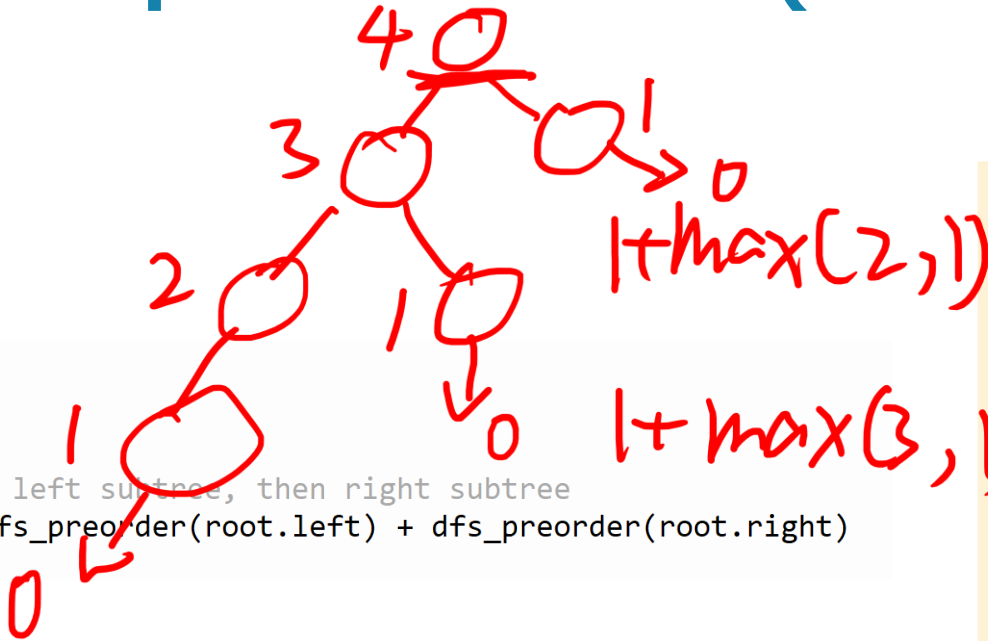
## DFS Graph-undirected



$V_1(\text{gray}) \rightarrow V_4(\text{gray}) \rightarrow V_3(\text{gray})$   
 $\rightarrow V_5(\text{gray}) \rightarrow V_2(\text{gray})$   
 $\rightarrow V_2(\text{black}) \rightarrow V_5(\text{black})$   
 $\rightarrow V_3(\text{black}) \rightarrow V_4(\text{black})$   
 $\rightarrow V_1(\text{black})$

# DFS implementation (Recursion)

```
def dfs_preorder(root):
    if not root:
        return []
    # visit current, then left subtree, then right subtree
    return [root.key] + dfs_preorder(root.left) + dfs_preorder(root.right)
```



**white** = undiscovered;  
**gray** = visited;  
**black** = finished/checked all neighbors.  
**pi** to record the node before current node  
**time** = global clock,  
**u.d** (discovered time) **u.f** (finished time)

```
DFS(G)
1 for each vertex  $u \in G.V$ 
2    $u.color = WHITE$ 
3    $u.\pi = NIL$ 
4  $time = 0$ 
5 for each vertex  $u \in G.V$ 
6   if  $u.color == WHITE$ 
7     DFS-VISIT( $G, u$ )
```

```
DFS-VISIT( $G, u$ )
1  $time = time + 1$  // white vertex  $u$  has just been discovered
2  $u.d = time$ 
3  $u.color = GRAY$ 
4 for each vertex  $v$  in  $G.Adj[u]$  // explore each edge  $(u, v)$ 
5   if  $v.color == WHITE$ 
6      $v.\pi = u$ 
7     DFS-VISIT( $G, v$ )
8  $time = time + 1$ 
9  $u.f = time$ 
10  $u.color = BLACK$  // blacken  $u$ ; it is finished
```

# DFS implementation (Recursion)

